

POSIX Threads (pthreads) — Beginner Exam Cheat Sheet

Concurrency · Thread Lifecycle · Creation · Joining · Critical Regions

1 · What is a Thread?

Process vs Thread — The Castle Analogy

- **Process** = a castle: isolated, safe, its own virtual address space (memory, files, sockets).
- **Thread** = a living creature *inside* the castle: shares all resources, runs independently.
- Each process starts with **one thread**; more can be created.

Formal Definitions

Process = virtual address space + file descriptors + *collection of threads*

Thread = lightweight execution unit that:

- Runs concurrently with other threads
- Shares process resources (memory, files, sockets)
- Has its **own stack**, registers, and program counter
- Is confined to one process — cannot migrate
- Can be created and destroyed independently

What Threads Share vs Own

SHARED (all threads)	PRIVATE (each thread)
Virtual address space	Stack
Global variables	Registers
Open files / sockets	Program counter
Heap memory	Thread ID (tid)
Code segment	Local variables
Signal handlers	errno value

■ **Castle metaphor:** *Threads are like servants — they roam freely, share toys, but can also break things!*

2 · Why Use Threads?

Use Case	How Threads Help	Example
Parallel computation	Run on multiple cores simultaneously	Matrix multiply rows in parallel
Overlap CPU + I/O	One thread computes while another waits	Download file while processing data
Async event handling	Handle multiple events concurrently	Web server: new request + ongoing transfer
Priority scheduling	Important tasks get CPU sooner	Real-time audio rendering
GUI responsiveness	Background work doesn't freeze UI	MS Word: type + auto-save + reformat

Thread models: **Pipeline** — chain of tasks each handled by a thread | **Manager/Worker** — one manager dispatches to worker pool | **Peer** — all threads including main do equal work

3 · Thread Lifecycle

Stage	How	Function
Creation	Created by another thread (or main)	pthread_create()
Running	Executes start_routine function	—
Waiting	Blocked on I/O, mutex, join, etc.	pthread_join(), etc.

Termination (voluntary)	Returns from start_routine OR calls exit function	pthread_exit()
Termination (by other)	Another thread requests cancellation	pthread_cancel()
Termination (all)	Any thread calls exit() or main() returns	exit()

■ ■ **Key Gotcha:** When main() returns or any thread calls exit(), the *entire process* and ALL threads die — even if other threads are still running!

4 · pthreads API — Function Reference Card

Function Signature	Returns	Purpose
pthread_create(thread, attr, start_fn, arg)	0=OK, >0=error	Create a new thread
pthread_exit(status)	never returns	Terminate calling thread
pthread_join(tid, &status)	0=OK, >0=error	Wait for thread to finish
pthread_cancel(tid)	0=OK, >0=error	Request thread cancellation
pthread_equal(t1, t2)	non-zero if equal	Compare two thread IDs
pthread_self()	pthread_t	Get own thread ID

Naming Conventions

Prefix	Applies To
pthread_	General thread functions
pthread_attr_	Thread attribute objects
pthread_mutex_	Mutex (lock) operations
pthread_mutexattr_	Mutex attribute objects
pthread_cond_	Condition variables
pthread_key_	Thread-specific data

■ **Compile flag:** cc -pthread myfile.c (links the pthreads library on Linux)

■ **pthreads functions do NOT set errno on errors — check the return value!**

5 · Essential Code Patterns

A) Thread Function (start_routine)

```
// Every thread needs a function like this:
void* my_thread_func(void* arg) {
    // Cast arg to your actual type:
    int* val = (int*) arg;
    // ... do work ...
    pthread_exit(NULL); // or return NULL;
}
```

- **Return type** must be void*
- **Parameter** must be void* (cast inside)
- Return value retrievable via pthread_join
- Never return a pointer to a **local (stack) variable!**

B) Creating a Thread

```
#include
pthread_t tid; // thread ID
int rc = pthread_create(
    &tid, // OUT: thread ID
    NULL, // attr: NULL = default
    my_thread_func, // function to run
    (void*) arg // argument to pass
);
if (rc != 0) {
    // handle error – rc is error code
}
```

C) Joining a Thread (Waiting for It)

```
void* ret_val;
int rc = pthread_join(tid, &ret_val);
// Blocks until thread 'tid' finishes
// ret_val = what thread passed to pthread_exit()
// Pass NULL as 2nd arg to ignore return value
```

- Only **one** thread may join a given thread
- Joining is how you **synchronize**: master waits for workers
- Equivalent of waitpid() for processes

D) Terminating a Thread

```
// Option 1: return from start_routine
void* my_func(void* arg) {
return NULL; // same as pthread_exit(NULL)
}
// Option 2: explicit exit anywhere
pthread_exit((void*) &result);
// Option 3: cancel from another thread
pthread_cancel(target_tid);
```

6 · Passing Arguments & Getting Results — Full Annotated Example

Step 1: Define a struct to bundle data for each thread

```
#include
#include
#include
#include
#define NUM_THREADS 8
// Use a struct to pass MULTIPLE values to a thread
struct thread_data {
int thread_num; // which thread is this?
char* message; // input data
int len; // output: result written here
};
```

Step 2: Write the thread function

```
void* PrintHello(void* threadarg) {
// Cast the void* back to the struct pointer
struct thread_data* my_data = (struct thread_data*) threadarg;
// Read input field
printf("Thread #%d: %s\n", my_data->thread_num, my_data->message);
// Write result INTO the struct (shared memory – no copy needed!)
my_data->len = strlen(my_data->message);
pthread_exit(NULL); // done
}
```

Step 3: main() — create threads, join them, read results

```
int main() {
pthread_t threads[NUM_THREADS];
struct thread_data data_array[NUM_THREADS]; // one element per thread
// -- FILL IN data for each thread --
for (int t = 0; t < NUM_THREADS; t++) {
data_array[t].thread_num = t;
data_array[t].message = "Hello!"; // set your input
// Pass POINTER to array element so thread can write results back
int rc = pthread_create(&threads[t], NULL,
PrintHello, &data_array[t]);
if (rc) { printf("Error: %d\n", rc); exit(-1); }
}
// -- JOIN (wait for) each thread --
int grand_total = 0;
for (int t = 0; t < NUM_THREADS; t++) {
pthread_join(threads[t], NULL); // wait until thread t finishes
grand_total += data_array[t].len; // now safe to read result
}
printf("Grand total = %d\n", grand_total);
pthread_exit(NULL); // end main thread (lets others finish if any)
}
```

Why pass <code>&data;_array[t]</code> not <code>&data;_array</code> ?	Each thread needs its OWN element. Passing the whole array would give every thread the same pointer.
Why NOT pass a loop variable's address?	The loop variable changes before the thread reads it — classic race condition bug!
Why use <code>pthread_exit(NULL)</code> in main?	It lets other threads finish instead of killing everything like <code>return 0</code> would.

7 · Concurrency: Single-Core vs Multi-Core

	Single-Core (Timesharing)	Multi-Core (True Parallelism)
How many threads run AT ONCE?	Only ONE at any time	One per core (can be >1 simultaneously)
When does switching happen?	Thread blocks on I/O / preempted by OS	OS schedules on multiple cores
Still useful with more threads than cores?	Yes! Threads waiting on I/O free the CPU	Yes! OS timeshares the extra threads
Architecture term	—	MIMD (Multiple Instructions, Multiple Data)

■ **Key insight:** Even on a single core, threads are useful because while one thread waits for I/O, another can compute. No CPU cycle is wasted!

8 · Race Conditions & Critical Regions

What is a Race Condition?

When two threads access **shared data** concurrently and the result depends on the unpredictable **order of execution**.

Example of the Problem

```
// Shared variable:
int x = 0;
// Thread 1 and Thread 2 BOTH run this:
x = x + 1;
// Assembly breakdown of x = x + 1:
// LOAD R1, x // read x into register
// ADD R1, 1 // increment
// STORE x, R1 // write back
// If both threads do LOAD before either STOREs
// both see x=0, both write x=1, NOT x=2!
```

■ **The result is WRONG because neither thread saw the other's write.**

4 Rules to Avoid Race Conditions

- **Mutual exclusion:** No two threads in their critical region at the same time.
- **No assumptions on speed** or number of CPUs.
- **Non-blocking outside:** A thread outside its critical region must not block others.
- **No starvation:** No thread waits forever to enter its critical region.

Critical Region = The Fix

The code that touches shared data is the **critical region**. Only one thread may execute it at a time.

The Lock Protocol

```
1. LOCK (grab the lock)
2. CRITICAL SECTION (touch shared data)
3. UNLOCK (release the lock)
// If another thread holds the lock,
// the waiting thread BLOCKS until released.
```

9 · User-Level vs Kernel-Level Threads

Feature	User-Level Threads	Kernel-Level Threads
Where managed?	By a runtime library in user space	By the OS kernel
Thread table lives in...	Each process (per-process)	The kernel (system-wide)
Context switch cost	Very fast (no kernel mode switch)	Slower (kernel mode switch required)
Blocking I/O problem	One blocked thread blocks ALL threads in process	Only that thread blocks
Preemption	Not possible unless thread yields voluntarily	OS can preempt any thread
Can use multiple cores?	No (kernel sees only 1 thread)	Yes
pthreads	Typically kernel-level on Linux	—

10 · Memory & Stack Limits

32-bit Address Space Constraints

- Linux 32-bit virtual address space: **2 GB**
- Default stack size per thread: **8 MB**
- 100 threads × 8 MB = **800 MB** just for stacks!
- Add heap (~200 MB) + executable (~50 MB) → near limit

Solutions

- Create **smaller stacks** (beware deep recursion!)
- Split work across **multiple processes** using pipes or shared memory
- Use a **64-bit OS** (address space = 2⁶⁴ bytes — virtually unlimited)

64-bit Math

```
Stack size = 8 MB = 2^23 bytes
Address space = 2^64 bytes
Usable (leave half for OS): 2^63
Max stacks = 2^63 / 2^23 = 2^40
≈ 1 TRILLION threads
(practically unlimited!)
```

■ On 64-bit Linux, stack memory is virtually unlimited. This is one of the main reasons modern systems use 64-bit.

11 · Common Exam Questions & Gotchas

■ Question / Scenario	■ Correct Answer / What to Write
What happens when main() returns with 10 other threads running?	The entire process terminates — ALL threads die. To avoid this, use pthread_exit(NULL) in main instead of return.
Why is status in pthread_join a void** (double pointer)?	Because we need to retrieve a void* value that the thread passed to pthread_exit(). A pointer to that void* is a void**.
What's wrong with: pthread_create(&t, NULL, fn, &i) inside a for loop?	Variable i changes before the thread reads it. Fix: copy i into a separate array element and pass &array[i].
Can a thread access global variables directly?	Yes — global variables live in shared memory. But concurrent access needs synchronization (locks) to avoid race conditions.
What is the difference between pthread_exit() and exit()?	pthread_exit() ends only the calling thread. exit() terminates the entire process and all its threads.
How do threads achieve concurrency on a single-core CPU?	Timesharing: OS preempts threads and context-switches between them. While one thread waits on I/O, another can run.
What must you NOT do with the return value of a thread function?	Do NOT return a pointer to a local (stack) variable. The stack is destroyed when the thread exits. Use heap or global memory.
Are newly created threads equal to the creating thread?	Yes — all threads are peers. There is no implied parent/child hierarchy. Any thread can join any other thread.

12 · Minimal Working Template — Memorize This!

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
// ① Define struct to hold per-thread input + output
typedef struct {
    int id; // input
    int result; // output (thread writes here)
} ThreadArg;
// ② Thread function — MUST match void* fn(void*) signature
void* worker(void* raw_arg) {
    ThreadArg* arg = (ThreadArg*) raw_arg; // ③ cast
    arg->result = arg->id * arg->id; // do work, write output
    return NULL; // same as pthread_exit(NULL)
}
int main() {
    #define N 4
    pthread_t tids[N];
    ThreadArg args[N];
    // ④ Create threads
    for (int i = 0; i < N; i++) {
        args[i].id = i;
        int rc = pthread_create(&tids[i], NULL, worker, &args[i]);
        if (rc) { perror("pthread_create"); exit(1); }
    }
}
```

```
// ⑤ Join threads + collect results
for (int i = 0; i < N; i++) {
    pthread_join(tids[i], NULL);
    printf("Thread %d: result = %d\n", i, args[i].result);
}
pthread_exit(NULL); // ⑥ end main – lets any remaining threads finish
}
```

✓ Checklist Item	Why it matters
#include <pthread.h>	Without this, nothing compiles
Compile with -pthread flag	Links the pthreads library
Thread fn signature: void* fn(void*)	API requirement — wrong signature = crash
Cast void* arg inside thread fn	Type safety — C won't do it automatically
Use struct array, not loop var address	Avoid race condition on argument passing
Check return value of pthread_create/join	Catch errors early (returns >0 on error)
pthread_exit(NULL) in main, not return	Keeps process alive until all threads done
Write results into shared struct	Safe way to return data from threads

■ Reference: ANSI/IEEE POSIX 1003.1-1995 | Compile: cc -pthread myfile.c | Good luck on your exam!